

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчёт

Лабораторная работа №2

Выполнил:

Карпов Данил Иванович

Группа J3210

ИСУ: 466130

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках второй лабораторной работы необходимо доработать сайт, выполненный в лабораторной работе №1, добавив взаимодействие с сервером. Статические данные, ранее вшитые напрямую в HTML, должны быть заменены на динамическую загрузку через HTTP-запросы к собственному API.

Требования к выполнению:

- Реализовать серверную часть — Mock API на базе JSON Server.
- Добавить авторизацию с использованием JWT-токенов:
 - Эндпоинты `POST /login` и `POST /register`
 - Хранение токена на клиенте в `localStorage`
 - Защита приватных маршрутов (проверка `Authorization: Bearer`)
- Подключить все страницы к API: загрузка курсов, данных пользователя, записей на курсы, сертификатов.
- Реализовать запись студента на курс через `POST /enrollments`.
- Реализовать создание нового курса преподавателем через `POST /courses`.

Ход работы

Реализован серверный модуль (**server.js**) на базе библиотеки JSON Server совместно с Express-middleware. Сервер запускается командой `npm start` и слушает порт 3001. База данных хранится в файле **db.json** и содержит коллекции: `users`, `courses`, `enrollments` и `certificates`.

Для авторизации подключены библиотеки `jsonwebtoken` и `bcryptjs`. Эндпоинт `POST /register` принимает имя, email, пароль и роль пользователя; пароль хешируется через `bcrypt.hashSync` перед сохранением. Эндпоинт `POST /login` проверяет хеш пароля и при совпадении возвращает подписанный JWT-токен со сроком жизни 24 часа.

Приватные эндпоинты (`/profile`, `/enrollments`, `POST /courses`) защищены middleware-функцией `verifyToken()`, которая извлекает токен из заголовка `Authorization: Bearer ...` и верифицирует его подпись. Неавторизованные запросы получают ответ `401 Unauthorized`.

Настроен CORS — сервер разрешает запросы с любого источника, что необходимо при разработке, когда фронтенд и бэкенд работают на разных портах.

На клиентской стороне создан общий модуль `js/auth.js`, подключаемый ко всем страницам. Модуль предоставляет функции: `login()`, `register()`, `logout()`, `isLoggedIn()`, `getCurrentUser()`, а также универсальный помощник `apiFetch()`, автоматически добавляющий заголовок `Authorization` к каждому запросу.

Функция `updateNavbar()` вызывается при загрузке каждой страницы и динамически переключает отображение: для неавторизованного пользователя показываются кнопки «Войти» и «Регистрация», для авторизованного — дропдаун с именем и аватаром.

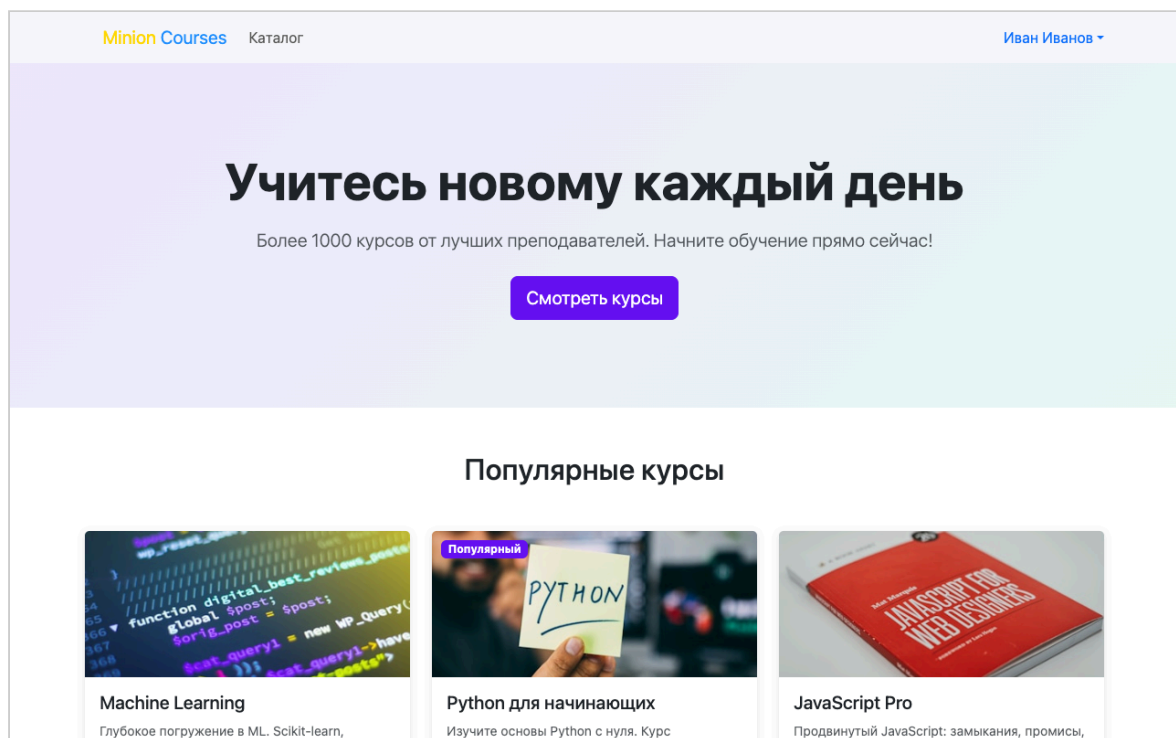
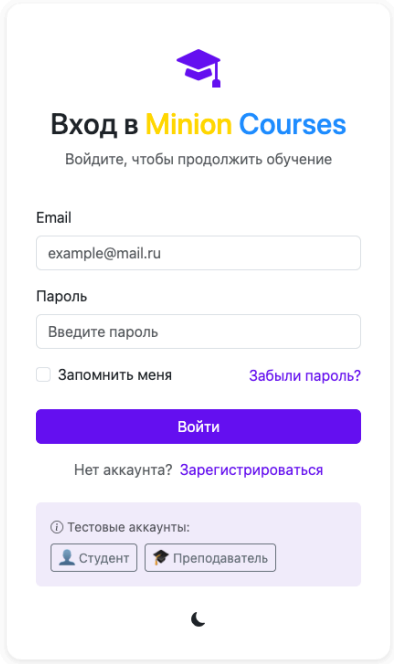


Рисунок 1 — Навбар с именем авторизованного пользователя

Формы входа и регистрации подключены к реальному API: при отправке выполняется `POST /login` или `POST /register`, полученный токен и объект пользователя сохраняются в `localStorage`. Если пользователь уже авторизован, страница автоматически перенаправляет его на главную.



The login form is centered on a light gray background. At the top is a purple graduation cap icon. Below it is the title "Вход в Minion Courses" in bold, with "Minion" in yellow and "Courses" in blue. A subtitle "Войдите, чтобы продолжить обучение" is in gray. The form includes an "Email" field with "example@mail.ru", a "Password" field with placeholder "Введите пароль", a "Remember me" checkbox, and a "Forgot password?" link. A purple "Войти" button is below. A link "Нет аккаунта? Зарегистрироваться" is next. A section for "Тестовые аккаунты:" shows "Студент" and "Преподаватель" buttons. A small black spinner is at the bottom.

Вход в **Minion Courses**

Войдите, чтобы продолжить обучение

Email

example@mail.ru

Пароль

Введите пароль

☐ Запомнить меня [Забыли пароль?](#)

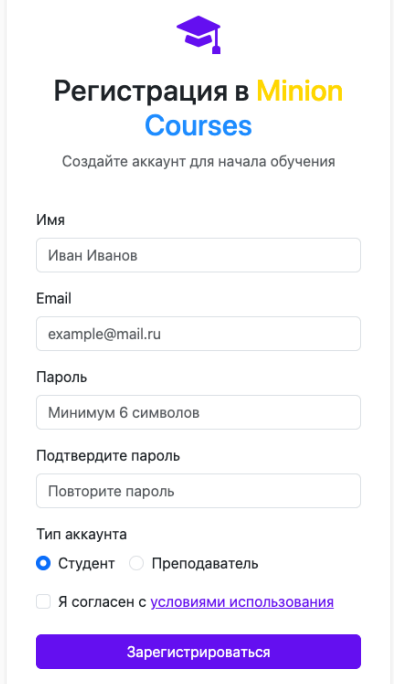
Войти

Нет аккаунта? [Зарегистрироваться](#)

Тестовые аккаунты:

⌂

Рисунок 2 — Страница входа с валидацией и спиннером



The registration form is centered on a light gray background. At the top is a purple graduation cap icon. Below it is the title "Регистрация в Minion Courses" in bold, with "Minion" in yellow and "Courses" in blue. A subtitle "Создайте аккаунт для начала обучения" is in gray. The form includes fields for "Имя" (Ivan Ivanov), "Email" (example@mail.ru), "Password" (with a "Минимум 6 символов" hint), and "Confirm password" (with placeholder "Повторите пароль"). It has radio buttons for "Студент" (selected) and "Преподаватель", a checkbox for "Я согласен с условиями использования", and a purple "Зарегистрироваться" button. A link "Уже есть аккаунт? Войти" is at the bottom.

Регистрация в **Minion Courses**

Создайте аккаунт для начала обучения

Имя

Иван Иванов

Email

example@mail.ru

Пароль

Минимум 6 символов

Подтвердите пароль

Повторите пароль

Тип аккаунта

☒ Студент ☐ Преподаватель

☐ Я согласен с [условиями использования](#)

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рисунок 3 — Страница регистрации

Главная страница и каталог курсов ([index.html](#), [search.html](#)) переведены на загрузку данных из API через GET `/courses`. На время ожидания ответа отображаются skeleton-плейсхолдеры; при сетевой ошибке — информационный алерт. Фильтрация в каталоге работает на стороне клиента по данным, уже загруженным из API.

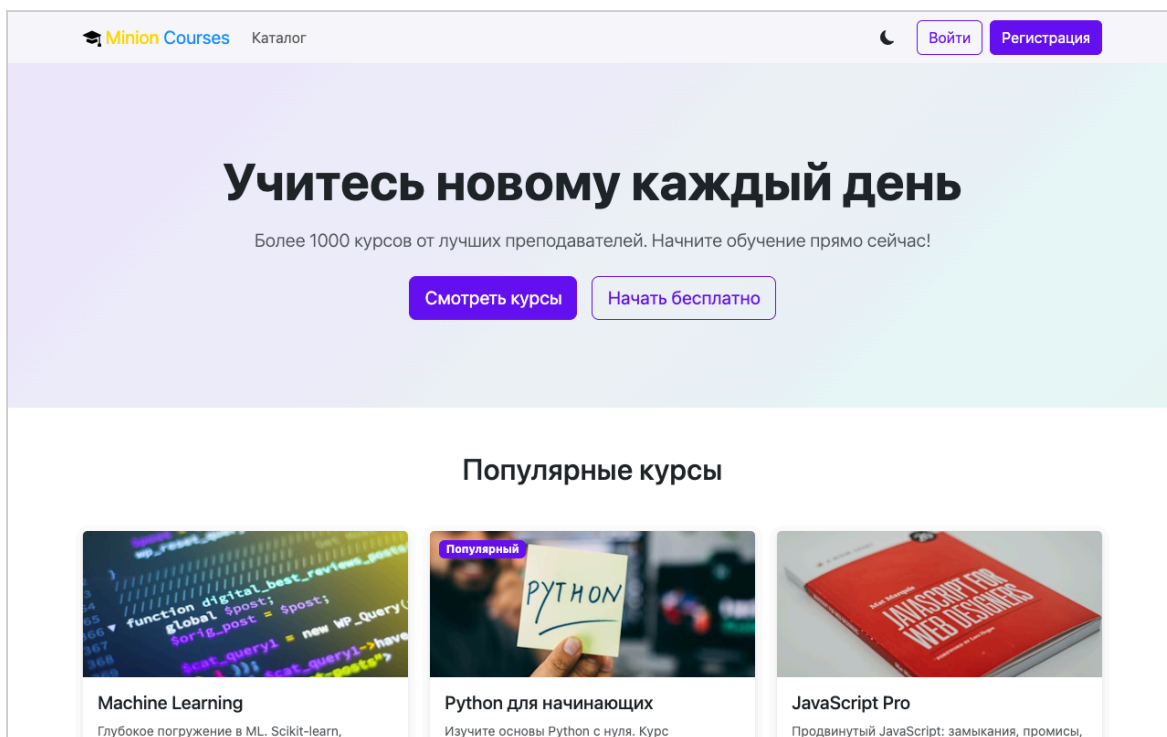


Рисунок 4 — Главная страница с курсами из API

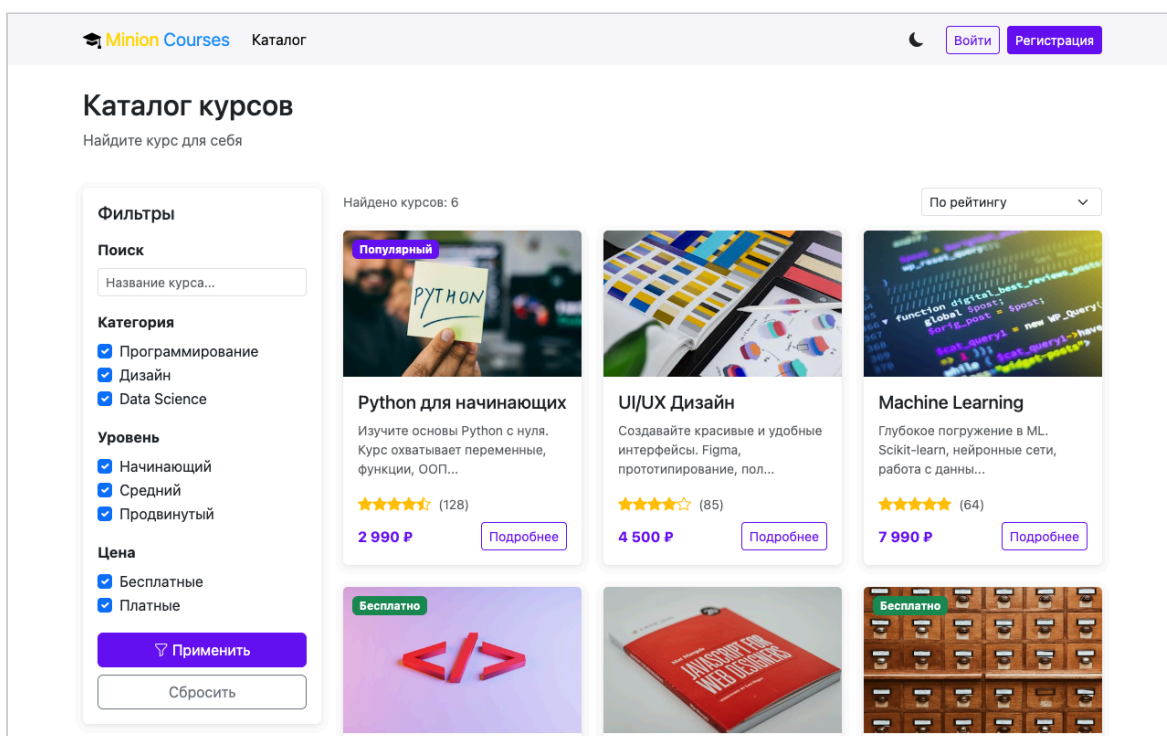
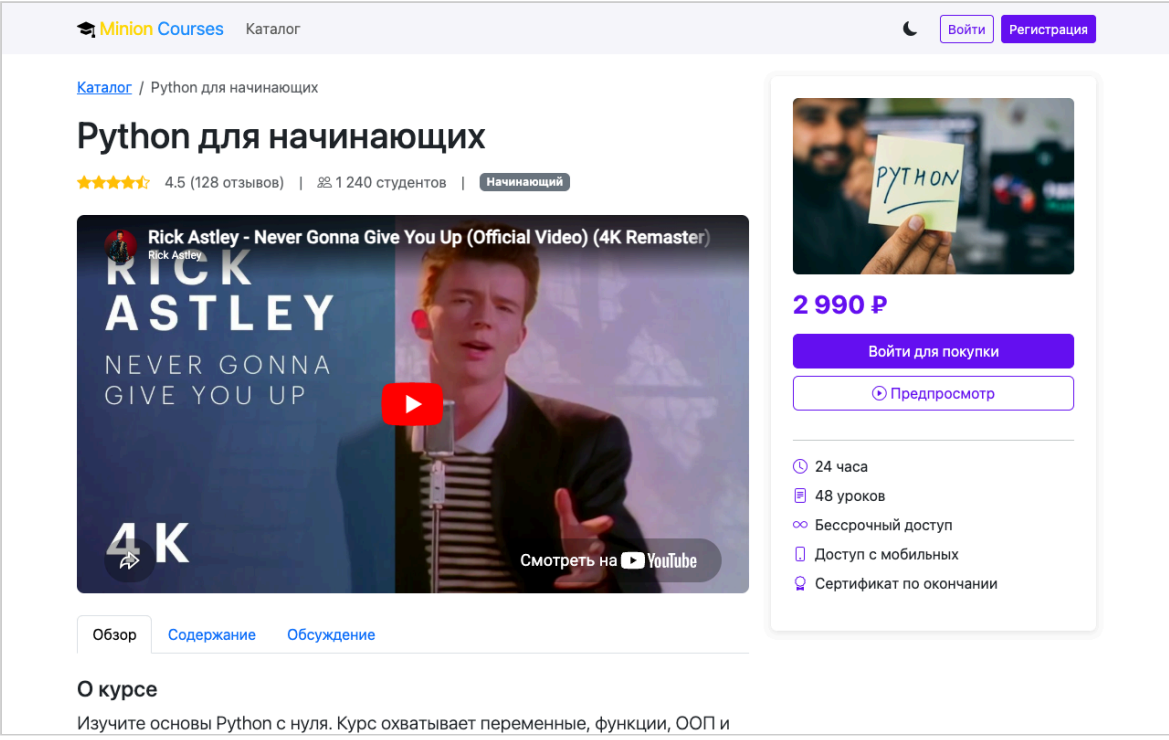


Рисунок 5 — Каталог с фильтрацией (данные из API)

Страница курса (**course.html**) загружает данные конкретного курса через GET `/courses/:id`. Добавлена кнопка «Записаться», при нажатии на которую выполняется POST `/enrollments` с передачей токена авторизации. Если пользователь уже записан на курс, кнопка заменяется на ссылку «Перейти к обучению». Неавторизованным пользователям предлагается войти в аккаунт.



The screenshot displays the 'Python для начинающих' (Python for Beginners) course page on the Minion Courses website. The page features a header with the site logo, a catalog link, and user options like 'Войти' (Login) and 'Регистрация' (Registration). The course title is prominently displayed, along with its rating (4.5 stars from 128 reviews) and the number of students (1,240). A video player shows Rick Astley's 'Never Gonna Give You Up' video. To the right, the course price is listed as 2,990 RUB, with buttons for 'Войти для покупки' (Login to purchase) and 'Предпросмотр' (Preview). Below the price, a list of course features is provided: 24 hours duration, 48 lessons, lifetime access, mobile access, and a certificate upon completion. The page also includes tabs for 'Обзор' (Overview), 'Содержание' (Content), and 'Обсуждение' (Discussion), and a section titled 'О курсе' (About the course) with a brief description.

Minion Courses Каталог

Каталог / Python для начинающих

Python для начинающих

★★★★☆ 4.5 (128 отзывов) | 1 240 студентов | Начинющий

Rick Astley - Never Gonna Give You Up (Official Video) (4K Remaster)

РICK ASTLEY

NEVER GONNA GIVE YOU UP

4K

Смотреть на YouTube

2 990 Р

Войти для покупки

Предпросмотр

- 24 часа
- 48 уроков
- Бессрочный доступ
- Доступ с мобильных
- Сертификат по окончании

Обзор Содержание Обсуждение

О курсе

Изучите основы Python с нуля. Курс охватывает переменные, функции, ООП и

Рисунок 6 — Страница курса с динамическими данными

Личный кабинет студента (**profile.html**) загружает профиль через GET /profile (защищённый эндпоинт), список записей через GET /enrollments?userId=... и сертификаты через GET /certificates?userId=... Прогресс каждого курса визуализируется компонентом Bootstrap Progress.

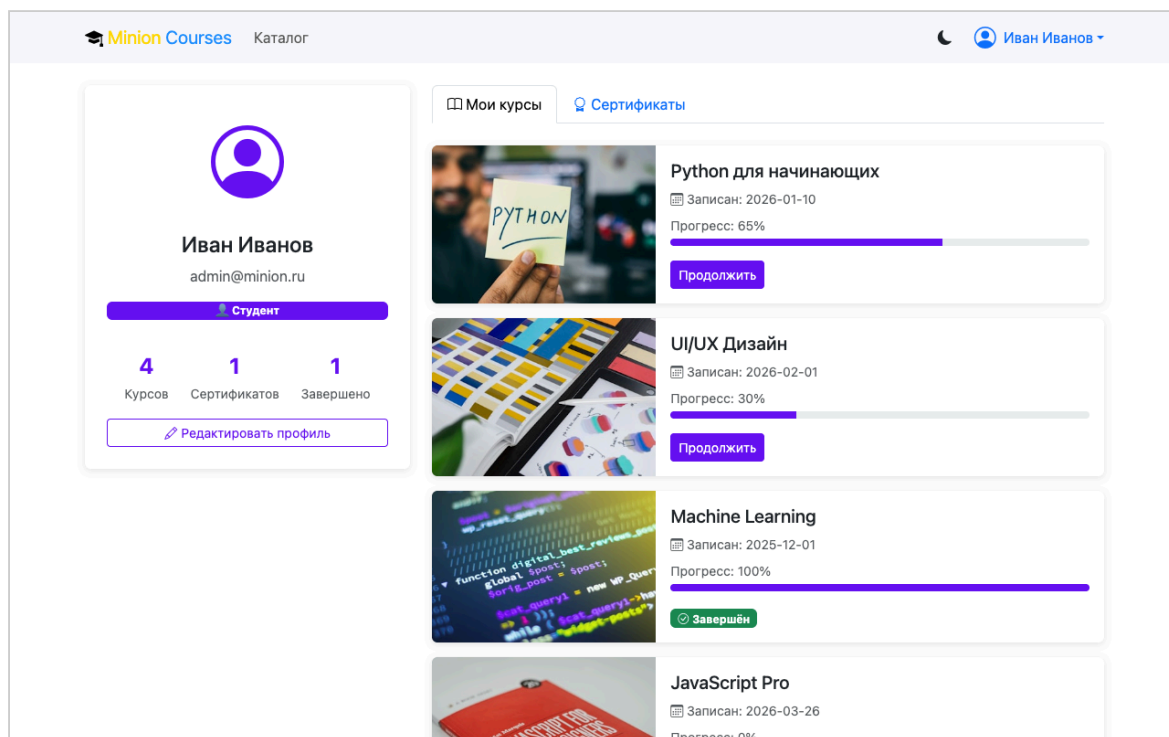


Рисунок 7 — Личный кабинет студента с данными из API

Личный кабинет преподавателя (**teacher.html**) загружает авторские курсы через GET `/courses?teacherId=...`. Для каждого курса отображается статистика: количество студентов (агрегируется из enrollments), средний рейтинг и цена. Кнопка «Создать курс» открывает модальное окно Bootstrap; при отправке формы выполняется POST `/courses` с JWT-токеном в заголовке. Новый курс немедленно появляется в списке без перезагрузки страницы.

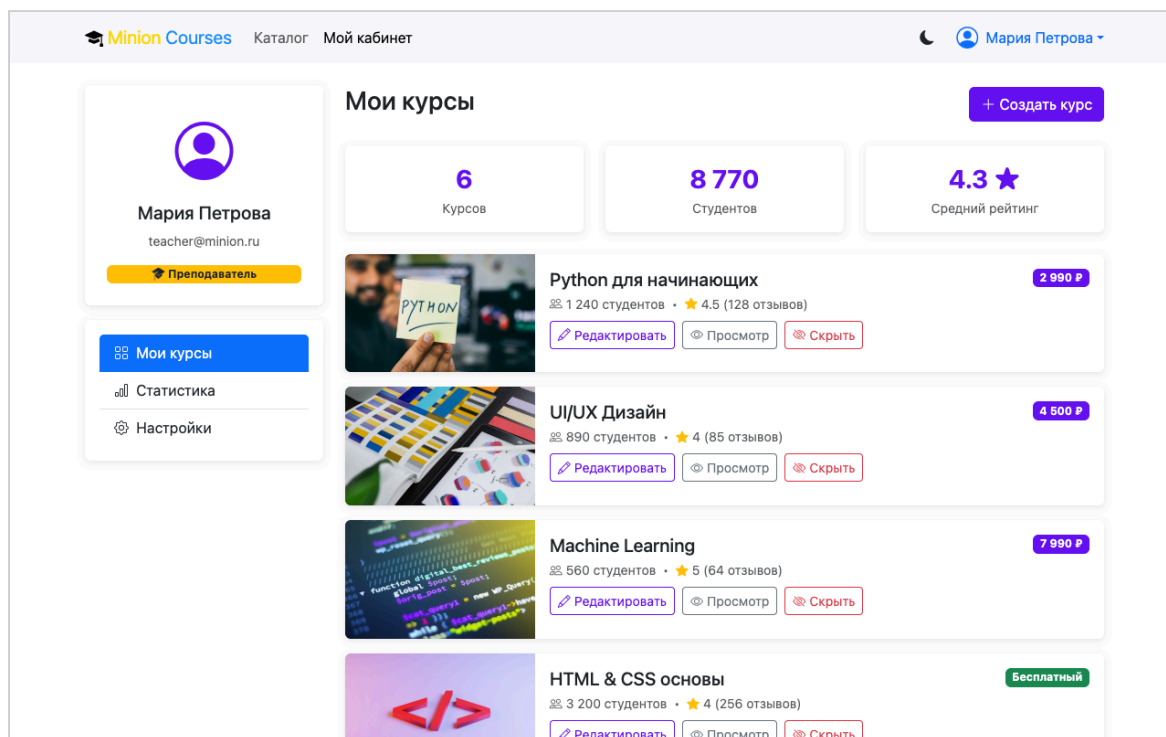


Рисунок 8 — Кабинет преподавателя с курсами из API

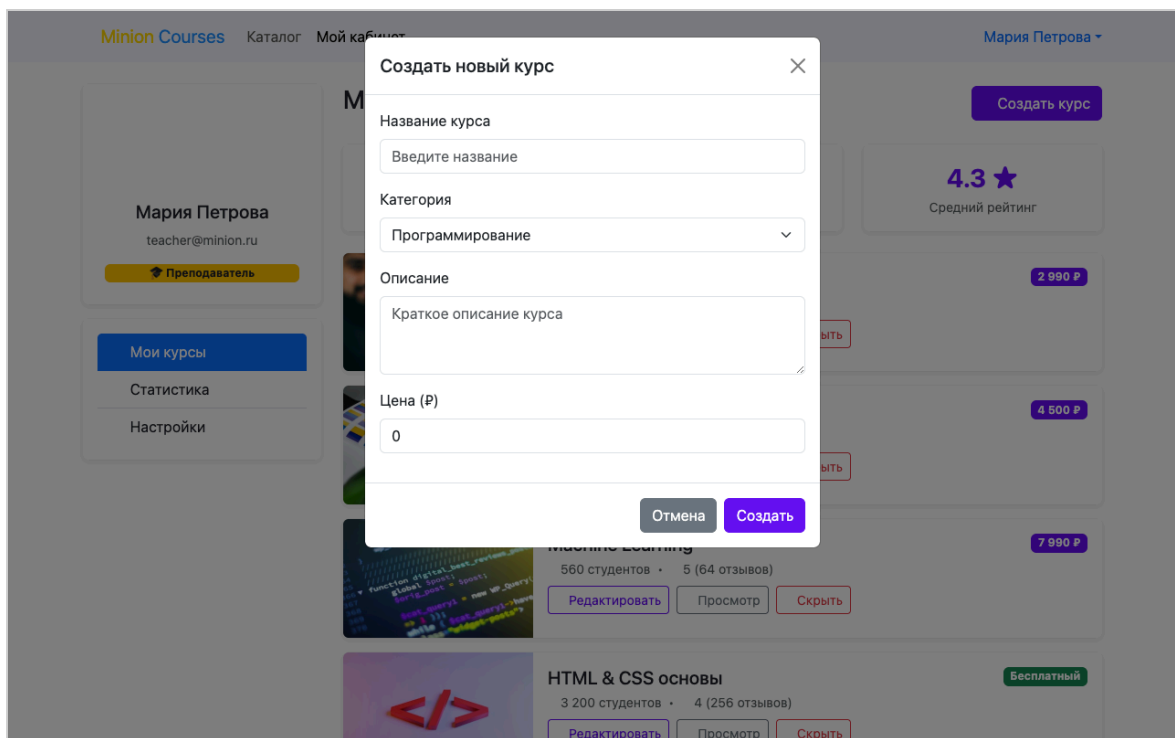


Рисунок 9 — Модальное окно создания нового курса

Вывод

В ходе выполнения лабораторной работы была реализована серверная часть приложения на базе JSON Server с кастомными Express-обработчиками, а также система авторизации с использованием JWT-токенов и хешированием паролей через bcrypt.

Освоены принципы построения REST API: разграничение эндпоинтов по методам HTTP (GET, POST, PATCH, DELETE), передача токена авторизации через заголовок **Authorization: Bearer**, защита приватных маршрутов на стороне сервера.

На клиентской стороне закреплены навыки работы с Fetch API: асинхронные запросы с **async/await**, обработка ошибок сети и HTTP-статусов, динамическое обновление DOM по данным из ответа сервера без перезагрузки страницы.

Реализован полноценный цикл аутентификации: регистрация нового пользователя, вход с получением токена, хранение сессии в **localStorage**, автоматическое добавление заголовка авторизации ко всем приватным запросам и выход с очисткой хранилища. Разработанное решение демонстрирует взаимодействие клиентской и серверной частей веб-приложения в соответствии с современными стандартами фронтенд-разработки.